File Upload Processing Service

-

Case Study

Prepared by

Vinod Balasubramanian

Table of Contents

Objective ***4***

Problem Statement ***4***

Possible Design Approaches ***4***

Event-Driven Queue Architecture **5**

Spring Batch-Centric Pipeline **5**

Hybrid Microservices Decomposition **5**

Direct Sync with Circuit Breaker (Baseline) **6**

Approach Taken [Assumption: 10K files/min with each file having 1M+ records] ***6***

C4 Model for the Approach Taken ***7***

Context Model **7**

Container Model **8**

Dataflow Model **8**

Deployment Architecture ***9***

Application Architecture ***10***

Scalability Mechanisms ***10***

Reliability & Fairness ***10***

Tech Stack Implementation ***11***

Epics ***11***

File Upload Module Service - Epic & User Stories **11**

Application Context 11

EPIC: Enterprise File Upload Processing Pipeline 11

Business Value 12

User Stories ***12***

Theme 1: File Upload & Security **12**

US-101: Secure File Upload Initiation 12
US-102: Enterprise Malware Scanning 12

Theme 2: File Validation & State Management 13
US-103: Dynamic Header Validation 13
US-104: Comprehensive File State Machine 13

Theme 3: Record Processing & Synchronization 13
US-105: Distributed Record Processing 13
US-106: External Endpoint Synchronization 14

Theme 4: Monitoring & Fairness 14
US-107: Multi-Level Status Tracking 14
US-108: Fairness & Priority Processing 14

Theme 5: Observability & Compliance 15
US-109: Production Observability 15
US-110: Regulatory Compliance & Audit 15

Architectural Characteristics Requirements 15

Success Metrics/KPIs 16

Implementation - Spec Driven Development 16

Constitution [.specify/memory/constitution.md] 16
Constitution for the CAMS File Processing Service 16
1.Mission Statement 16
2. Core Architectural Principles 17
3. The Architectural Blueprint 17
4. Approved Technology Stack 18
5. Non-Functional Requirements (Mandatory Checks) 19

Specify [./specify/specs/*.md] 19
Theme 1: File Upload & Security 19
Theme 2: File Validation & State Management 19
Theme 3: Record Processing & Synchronization 19
Theme 4: Monitoring & Fairness 19
Theme 5: Observability & Compliance 20

US-101: Secure File Upload Initiation 20
Specification [specs/001-secure-file-upload/spec.md] 23
Research file [specs/001-secure-file-upload/research.md] 25
DataModel [specs/001-secure-file-upload/data-model.md] 26
Plan [specs/001-secure-file-upload/plan.md] 28

Tasks [specify/001-secure-file-upload/tasks.md] 30

Questions to Siva ***35***

Objective

The objective of this document is to capture how the high-level epic to handle file upload is getting translated into design and implementation and showcase how AI is used to create them.

Problem Statement

Design a file upload module. Each file will have at least a million entries

• Should include Malware scanning

• Include various file processing status (state machine) for file upload tracking

• Includes file header validation against a dynamic template. Depending on flow type a template will be used to validate

• Post file validation, do record level processing and support tracking record

level status

• Records will be processed against an external endpoint for processing in a

synchronous fashion

Support the design document with components for scalability, reliability and fairness.

Possible Design Approaches

Several architectural designs can handle the high-throughput file upload microservice requirements, focusing on scalability via GCP services, reliability through async processing and retries, and fairness with queues and chunking. These leverage Spring Boot, Spring Batch, Spring State Machine for status tracking, ClamAV or API-based malware scanning, and GitHub SpecKit for spec-driven development.

Event-Driven Queue Architecture

Files from UI, SFTP (via Spring Integration), or GCS (via notifications) trigger Pub/Sub messages with metadata, enabling stateless upload endpoints that scale horizontally on Cloud Run or GKE.

- Queue ensures fairness (FIFO or priority) and reliability (retries, dead-letter queues).
- Dedicated processors consume queues: scan with ClamAV daemon or API (e.g., attachmentAV), validate headers against templates, then use Spring Batch for chunked record processing against REST endpoints.
- State machine persists file/record statuses in Firestore or Spanner; record-level tracking uses partitioned IDs for parallelism.
  This handles 10K files/min (~167/sec) by auto-scaling workers, avoiding sync bottlenecks.

Spring Batch-Centric Pipeline

Use Spring Batch as core for all batch sources: SFTP/GCS polling or events launch jobs with chunk-oriented processing (e.g., 1K records/chunk) for header validation, scanning, and REST calls.

- UI uploads stream directly to GCS, triggering Batch jobs via Cloud Scheduler or Functions.
- JobRepository in Spanner tracks states; Spring State Machine embeds for file/record FSM (e.g., UPLOADED → SCANNED → VALIDATED → PROCESSING → COMPLETED).
- Scale via GKE Horizontal Pod Autoscaler; fairness from job partitioning and skip/retry policies.
  Reliable for massive records (million/file) with fault-tolerant chunking, but less reactive than pure events.

Hybrid Microservices Decomposition

Break into focused services: Upload Service (UI/SFTP/GCS ingress), Scan Service (ClamAV), Validator Service (header templates), Record Processor Service (Spring Batch chunks to REST), Tracker Service (state machine).

- Async via Pub/Sub or Kafka on GCP for inter-service comms; store files in GCS for shared access.

- Each scales independently (e.g., more record processors); fairness via consumer groups and backpressure.
- React frontend polls Tracker for statuses; deploy all on GKE with Istio for traffic management.
  Maximizes reliability through isolation, suits Java/Spring stack.

Direct Sync with Circuit Breaker (Baseline)

Synchronous flow in single service: receive file, stream-scan-validate-process, but unrealistic for volume—instead, use as fallback with Resilience4j circuit breakers on REST calls and async offload for records.

- Not recommended primarily due to throughput limits, but viable for low-volume UI with queue fallback.
  Use only if simplifying for MVP, then evolve to async.

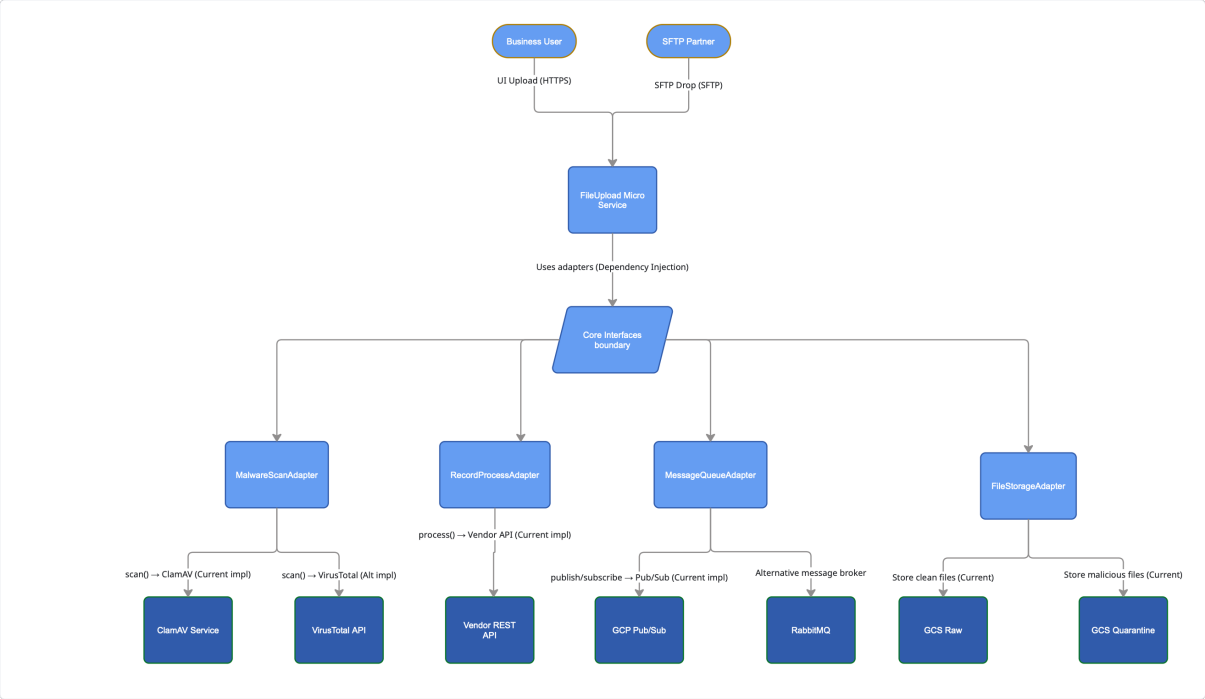
Design	Scalability	Reliability	Fairness	Complexity
Event-Driven Queue	High (Pub/Sub + GKE autoscaling)	High (DLQ, retries)	High (queues)	Medium
Spring Batch-Centric	Medium-High (job scaling)	High (built-in retry)	Medium (partitioning)	Low
Hybrid Decomposition	Highest (per-service)	Highest (isolation)	High (consumer groups)	High
Direct Sync	Low	Low-Medium	Low	Lowest

Approach Taken [Assumption: 10K files/min with each file having 1M+ records]

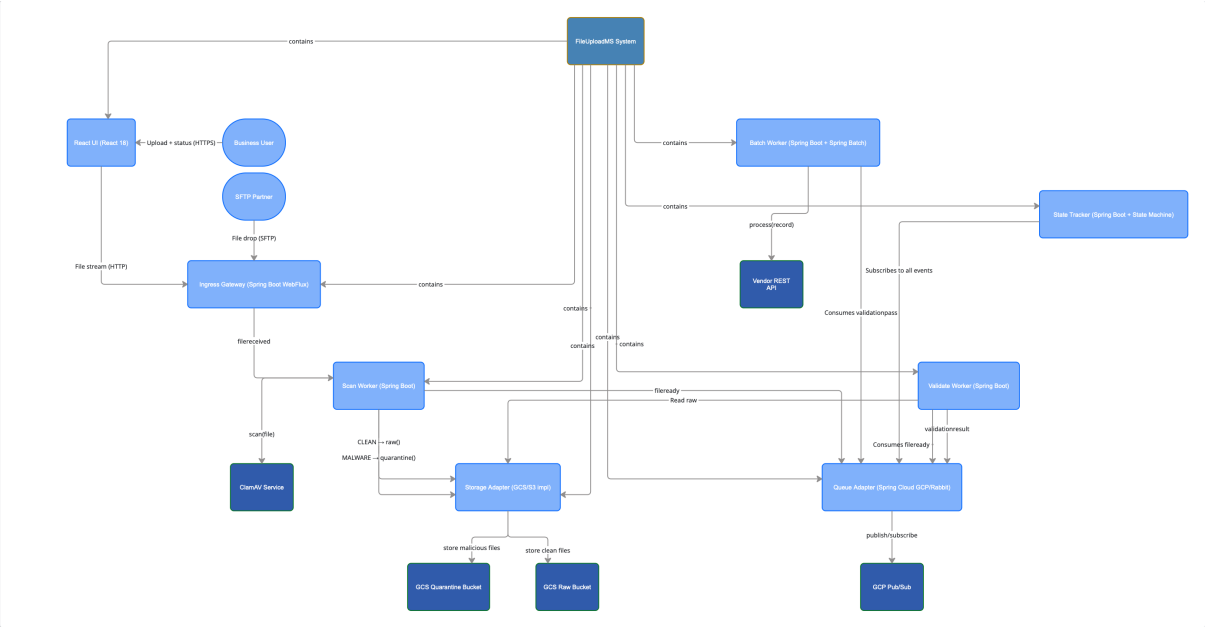
Event-driven with queues best fits for the given use case. Will use SpecKit for BDD specs on workflows (e.g., file states, validation).

C4 Model for the Approach Taken

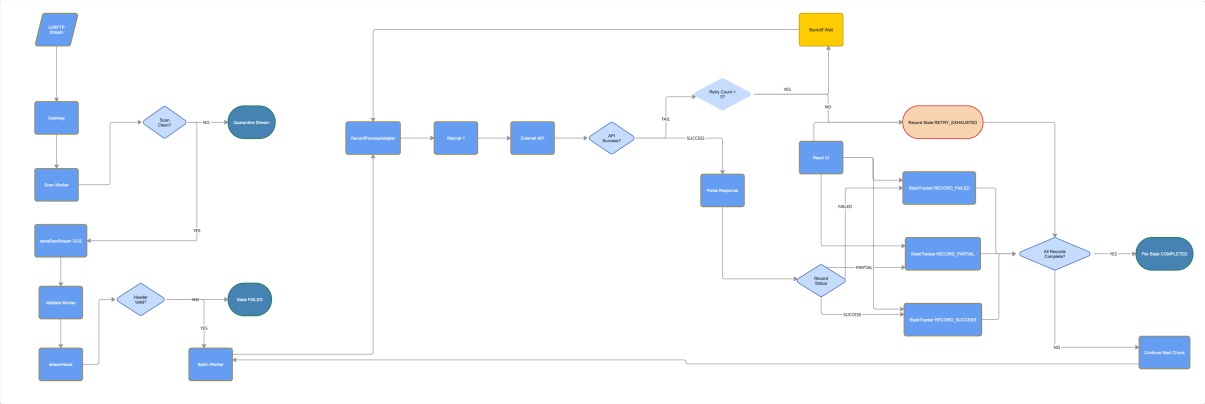
Context Model



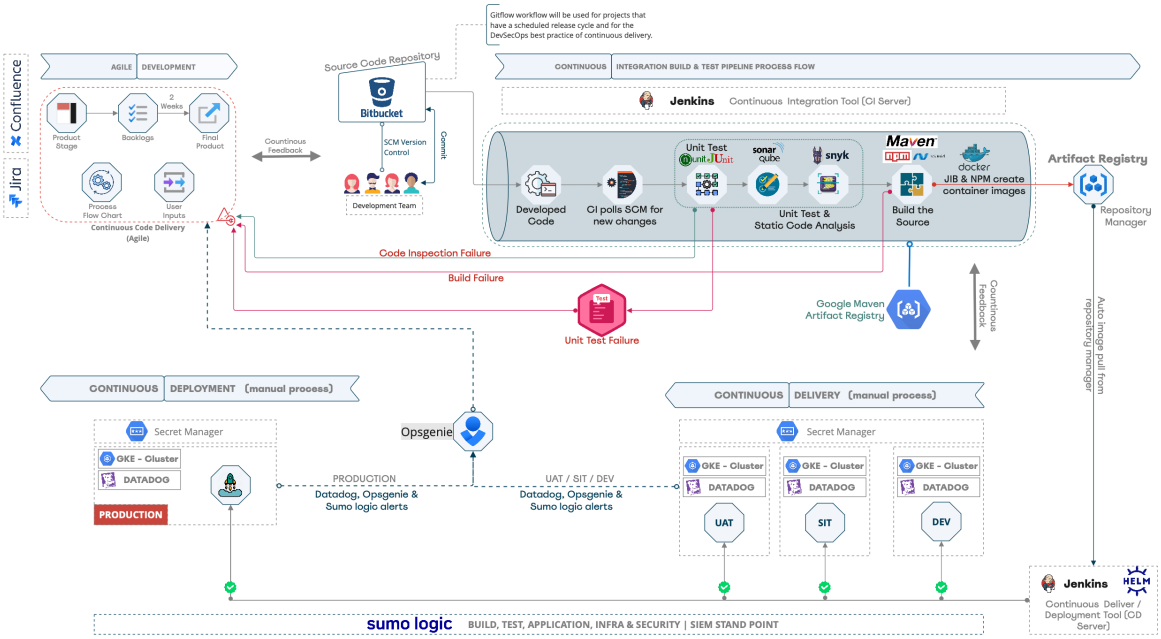
Container Model



Dataflow Model

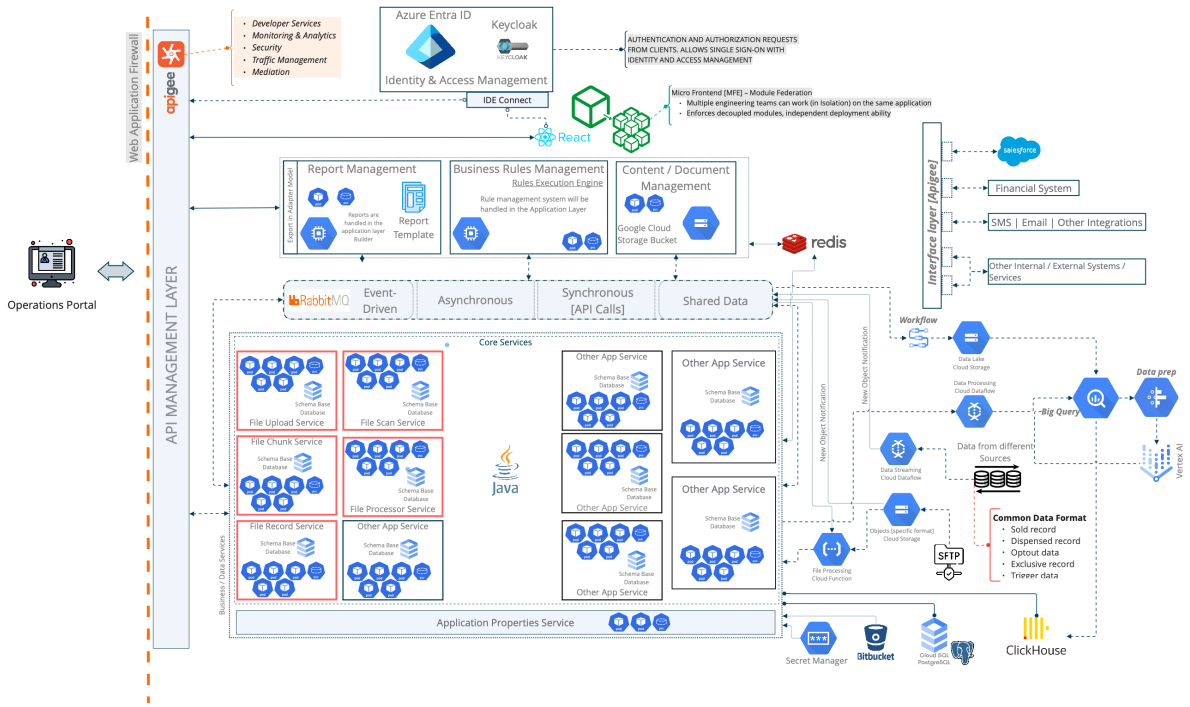


Deployment Architecture



After the discussion with Siva we have to update them based on the approach we have taken for the existing platform engineering setup. This is more about how we leverage existing GCP environment and its native services as much as possible and use many of the open source software and manage them internally to avoid additional cost.

Application Architecture



After the discussion with Siva we have to update them based on the approach we have taken for the existing application setup. This is more about how we leverage existing GCP environment and its native services as much as possible and see how the transactional and analytical systems could together in the grand scheme of CAMS AI and showcase [in PINK Box] the services that will be created for our use case.

Scalability Mechanisms

- Horizontal scaling: GKE HPA (CPU/concurrency-based) for workers; Pub/Sub auto-scales to millions msgs/sec.
- Parallelism: Partition Pub/Sub by fileId hash; Spring Batch multi-threaded chunks (e.g., 100 threads/file).
- Backpressure: Subscription flow control, max 1K unacked msgs/worker.

Reliability & Fairness

- At-least-once delivery + idempotency (fileId/recordId keys).
- Retries/DLQ: Pub/Sub dead-letter topics → manual requeue.
- State Machine persistence: Every transition writes to Spanner (strong consistency for queries).
- Fairness: FIFO within fileId partitions; priority queues for UI vs batch sources.

Tech Stack Implementation

Layer	Tech	Why
Core MS	Spring Boot 3.x + WebFlux (reactive for streams)	Async I/O for high throughput
Queue	GCP Pub/Sub + Spring Cloud GCP	Native integration, ordering
Batch	Spring Batch + Partitioning	Chunked record processing
State	Spring State Machine + Spanner	File/record FSM persistence
Scan	ClamAV Docker sidecar or VirusTotal API	Lightweight, integrable
Frontend	React + Axios polling/WebSocket	Status dashboard
Deploy	GKE + Istio	Autoscaling, traffic splitting
Specs	GitHub SpecKit + JUnit5	BDD for workflows

Epics

File Upload Module Service - Epic & User Stories

Application Context

Cams Online (Asset Management) - Processing **1M+ record files** (NAV, Transaction Ledgers, Portfolio Updates) with regulatory compliance [PCI-DSS, SEBI].

EPIC: Enterprise File Upload Processing Pipeline

As a Cams Operations Manager, **I want** a secure, scalable file upload microservice that processes 1M+ record financial files with malware scanning, validation, and external synchronization so that **regulatory reporting and portfolio updates occur without data loss or security incidents**.

Business Value

₹50Cr+ daily transaction volume processing, 99.99% uptime, zero security incidents.

User Stories

Theme 1: File Upload & Security

US-101: Secure File Upload Initiation

As a Portfolio Manager, **I want** to upload 1M+ record files via signed URLs so that **files are securely received without size limits**.

Acceptance Criteria:

- Support CSV/JSON files up to 5GB (1M+ records)
- Generate pre-signed upload URLs (24hr expiry)
- Extract file metadata (type, flow_type, checksum)
- Store raw files in isolated bucket with TTL
- API response < 200ms (p99)

US-102: Enterprise Malware Scanning

As a Security Officer, **I want** automatic malware scanning on all uploads so that **no infected files enter the organisation**.

Acceptance Criteria:

- Scan completion < 5min (p95) for 250MB files
- Multi-engine scanning (ClamAV + commercial)
- Quarantine infected files with forensic metadata
- Alert security team + reject upload (<30s notification)
- 100% false positive review workflow

Theme 2: File Validation & State Management

US-103: Dynamic Header Validation

As a Operations Lead, **I want** file header validation against Cams-specific templates so that **only correctly formatted files proceed**.

Acceptance Criteria:

- Support 12 Cams file templates (NAV, Transactions, Holdings)
- Auto-select template by file_type field
- Validate headers, record count, data types, checksums
- Reject malformed files with detailed error report
- Template versioning (v1.2 → v2.0 migration)

US-104: Comprehensive File State Machine

As a Business User, **I want** real-time file processing status so that **I can track 1000+ concurrent file uploads**.

Acceptance Criteria:

- 12 distinct file states (RECEIVED → ARCHIVED)
- Real-time status API (WebSocket polling)
- Dashboard showing queue position, ETA
- Email/SMS alerts on state transitions
- Historical audit trail (immutable)

Theme 3: Record Processing & Synchronization

US-105: Distributed Record Processing

As a Data Engineer, **I want** parallel record processing (1000 records/batch) so that **1M records complete in <2hrs**.

Acceptance Criteria:

- Horizontal scaling to 100+ worker instances
- Batch processing (1000 records per external API call)
- Exactly-once processing guarantee
- Progress tracking (847K/1M processed)

- Dynamic backpressure (95% CPU threshold)

US-106: External Endpoint Synchronization

As a Integration Architect, **I want** reliable synchronous calls to external NAV/Portfolio APIs so that **record updates are transactionally safe**.

Acceptance Criteria:

- Circuit breaker pattern (5xx failures)
- Exponential backoff (2^n minutes, max 3 retries)
- Idempotency via record UUID + hash
- Dead letter queue for permanent failures
- 99.9% sync success rate

Theme 4: Monitoring & Fairness

US-107: Multi-Level Status Tracking

As a Compliance Officer, **I want** file + record level status tracking so that **I can audit processing for RBI/SEBI**.

Acceptance Criteria:

- File status: RECEIVED | SCANNING | PROCESSING | COMPLETED
- Record status: PENDING | SYNCING | SYNCED | FAILED
- Sharded NoSQL schema (Firestore/DynamoDB)
- Global secondary indexes for reporting
- Retention: 7yr compliance archive

US-108: Fairness & Priority Processing

As a Operations Manager, **I want** priority queues for urgent files so that **NAV cutoffs are never missed**.

Acceptance Criteria:

- 3 priority tiers (P0-NAV, P1-Transactions, P2-Bulk)
- Weighted fair queuing (P0:50%, P1:30%, P2:20%)
- SLA monitoring (P0 <15min end-to-end)
- Queue depth dashboard + alerts

- Prevent starvation (P2 max wait 4hrs)

Theme 5: Observability & Compliance

US-109: Production Observability

As a SRE, **I want** comprehensive monitoring so that **I can maintain 99.99% uptime**.

Acceptance Criteria:

- Distributed tracing (file → record → external API)
- Custom metrics (files/min, records/sec, error rates)
- SLO dashboards (5min PROCESSING → alert)
- Log aggregation with structured JSON
- Chaos engineering readiness

US-110: Regulatory Compliance & Audit

As a Compliance Manager, **I want** complete audit trails so that **RBI/SEBI inspections pass without issues**.

Acceptance Criteria:

- Immutable processing logs (IPFS/blockstore)
- File checksum validation at every stage
- Data lineage (file → record → external system)
- PII redaction in logs
- Export to compliance SIEM (Splunk/QRadar)

Architectural Characteristics Requirements

Category	Requirement	Target
Scale	Concurrent files	1000+
Scale	Records/sec	10K+
Latency	File-to-complete	<2hrs (p95)
Availability	Uptime	99.99%

Category	Requirement	Target
Security	False negatives	0%
Cost	Per 1M file	<$2.00
Compliance	Audit retention	7 years

Success Metrics/KPIs

- 99.99% files processed without data loss
- 95% files complete <2hrs
- Zero malware incidents
- 100% regulatory audit pass rate
- Cost <₹150/file at scale

Implementation - Spec Driven Development

Now that we have the epics and user stories created we will be using Specification Driven Development using Github Speckit. This helps you to create the constitution to capture the mission, core architectural principles, architectural blueprint, approved technology stack, NFRs. Once we create the constitution we will create the specs and come up with the plan and tasks for the plan and proceed with the implementation to build the Frontend and Backend applications and the application, infrastructure and security scaffolding.

Constitution [.specify/memory/constitution.md]

Constitution for the CAMS File Processing Service

1.Mission Statement

This document outlines the foundational principles, architectural commitments, and technical standards for the CAMS File Processing Service. As the AI programming assistant, I will adhere to this constitution to ensure all generated code, advice, and modifications align with the project's strategic goals of scalability, reliability, security, and compliance.

2. Core Architectural Principles

Event-Driven First: *The system will be built on an event-driven, queue-based architecture. All long-running processes (scanning, validation, record processing) must be decoupled and triggered by asynchronous messages, preferably via GCP Pub/Sub. This is the primary pattern for ensuring scalability and resilience.*

Asynchronous Processing is Mandatory: *No synchronous, blocking operations are permitted for file processing tasks. The initial file upload will be offloaded directly to GCS via pre-signed URLs. The API will respond immediately, and the workflow will proceed in the background.*

Horizontal Scalability by Design: *All processing components (workers) must be stateless and designed to scale horizontally. We will leverage Google Kubernetes Engine (GKE) with Horizontal Pod Autoscaling (HPA) as the primary deployment target.*

Reliability Through Idempotency and State: *Every process must be idempotent to support at-least-once message delivery. State transitions must be persisted transactionally to a durable database (Spanner) using the Spring State Machine pattern. Dead-letter queues (DLQs) are required for handling terminal failures.*

Spec-Driven Development: *The API contract is king. Development will follow a spec-driven approach using OpenAPI. All API changes must begin with a modification to the openapi.yaml specification.*

3. The Architectural Blueprint

I will follow the "Event-Driven Queue Architecture" as the canonical design.

Ingress: *A REST API (File Upload Service) provides a pre-signed URL for direct GCS upload. It does not handle file streams. Upon client confirmation of upload, it publishes a FILE_UPLOADED event to Pub/Sub.*

Storage: *GCS is the source of truth for files, segregated into quarantine, clean, and failed buckets.*

Messaging: *GCP Pub/Sub is the central nervous system for orchestrating the workflow between microservices.*

Processing Workers:

- **Scan Service**: A dedicated worker (or Cloud Function) listens for new files, performs malware scanning, moves the file, and publishes the result.
- **Validation Service**: A worker consumes SCANNED_CLEAN events, validates file headers against dynamic templates stored in the database, and publishes the result.

- **Record Processing Service**: A worker consumes VALIDATION_SUCCESS events, uses Spring Batch to process records in partitioned chunks, calls the external synchronous endpoint, and tracks record-level status.

State Management:

- **File State**: A FileStatus entity will be managed by a Spring State Machine, with its state persisted in Cloud Spanner.
- **Record State**: A RecordStatus entity will track the status of individual records, also in Spanner, to support audit and reporting requirements.
- **Frontend**: The React frontend will poll a Tracker Service API for file and record-level status updates.

4. Approved Technology Stack

I will generate code and provide solutions using only the following approved technologies:

Layer	Technology	Rationale
Core	Spring Boot 3.x + WebFlux	High-throughput, non-blocking I/O.
Queue	GCP Pub/Sub + Spring Cloud GCP	Native, scalable, and integrated messaging.
Batch	Spring Batch	Robust, fault-tolerant chunk processing.
State	Spring State Machine + Cloud Spanner	Transactional FSM persistence, strong consistency.
Scan	ClamAV (Docker sidecar) or VirusTotal API	Integrable and standard security scanning.
Deploy	GKE + Istio	Managed Kubernetes for autoscaling and mesh.
Specs	OpenAPI 3.0 (SpecKit) + JUnit 5	Contract-first development and testing.
Frontend	React	For the status dashboard UI.

5. Non-Functional Requirements (Mandatory Checks)

I must ensure my suggestions and code adhere to these critical requirements:

- **Security**: All uploads must be scanned for malware. PII must be redacted from logs. File integrity must be verified with checksums at key stages.
- **Compliance**: An immutable audit trail for every file and record state transition must be maintained for 7 years. Data lineage must be traceable.
- **Performance**: The end-to-end processing time for a 1M record file must target **< 2 hours (p95)**. API endpoints must respond in **< 200ms (p99)**.
- **Fairness**: The system must support priority queues (e.g., P0 for NAV files) to prevent starvation and meet SLAs.
- **Observability**: Code must include support for distributed tracing, structured logging (JSON), and custom metrics for monitoring key business operations (e.g., files/min, record sync rate).
- **Reliability**: All external calls must be wrapped in a Circuit Breaker (Resilience4j) with exponential backoff and retry policies.

This constitution is the source of truth for my behavior on this project. Any deviation must be explicitly requested and justified.

Specify [./specify/specs/*.md]

Theme 1: File Upload & Security

- /speckit.specify US-101: Secure File Upload Initiation
- /speckit.specify US-102: Enterprise Malware Scanning

Theme 2: File Validation & State Management

- /speckit.specify US-103: Dynamic Header Validation
- /speckit.specify US-104: Comprehensive File State Machine

Theme 3: Record Processing & Synchronization

- /speckit.specify US-105: Distributed Record Processing
- /speckit.specify US-106: External Endpoint Synchronization

Theme 4: Monitoring & Fairness

- /speckit.specify US-107: Multi-Level Status Tracking

- /speckit.specify US-108: Fairness & Priority Processing

Theme 5: Observability & Compliance

- /speckit.specify US-109: Production Observability
- /speckit.specify US-110: Regulatory Compliance & Audit

US-101: Secure File Upload Initiation

For the first user story we will run the /speckit.specify US-101: Secure File Upload Initiation command after giving the context and get the checklist [requirements.md], contracts[openapi.yaml], data-model, plan, quickstart, research, spec and tasks markdown files.

Requirements [/specs/001-secure-file-upload/checklists/requirements.md]

Specification Quality Checklist: US-101: Secure File Upload Initiation

Purpose: Validate specification completeness and quality before proceeding to planning **Created**: 2026-03-08 **Feature**: spec.md

Content Quality

- [X] No implementation details (languages, frameworks, APIs)
- [X] Focused on user value and business needs
- [X] Written for non-technical stakeholders
- [X] All mandatory sections completed

Requirement Completeness

- [X] No [NEEDS CLARIFICATION] markers remain
- [X] Requirements are testable and unambiguous
- [X] Success criteria are measurable
- [X] Success criteria are technology-agnostic (no implementation details)
- [X] All acceptance scenarios are defined
- [X] Edge cases are identified
- [X] Scope is clearly bounded
- [X] Dependencies and assumptions identified

Feature Readiness

- [X] All functional requirements have clear acceptance criteria
- [X] User scenarios cover primary flows
- [X] Feature meets measurable outcomes defined in Success Criteria
- [X] No implementation details leak into specification

Notes

- All checks passed. The specification is ready for the next phase.

Openapi.yaml

```yaml
openapi: 3.0.3
info:
  title: CAMS File Upload Service
  description: |-
    API for initiating and tracking file uploads in the CAMS processing pipeline.
  version: 1.0.0
servers:
  - url: https://api.cams.com/v1
    description: Production Server
  - url: http://localhost:8080/api/v1
    description: Local Development Server
tags:
  - name: FileUpload
    description: Operations for file upload initiation and status tracking.

paths:
  /files/upload-request:
    post:
      tags:
        - FileUpload
      summary: Request a pre-signed URL for file upload
      description: |-
        Initiates the file upload process. The client provides metadata about the file,
        and the service returns a unique `fileId` and a secure, time-limited URL
        to which the file can be uploaded directly.
      operationId: requestUploadUrl
      requestBody:
        description: Metadata for the file to be uploaded.
        required: true
        content:
          application/json:
            schema:
              $ref: '#/components/schemas/UploadRequest'
      responses:
        '200':
          description: Successfully generated upload URL.
          content:
```

```yaml
          application/json:
            schema:
              $ref: '#/components/schemas/UploadResponse'
      '400':
        description: Bad Request. The request is missing required fields or is malformed.
        content:
          application/json:
            schema:
              $ref: '#/components/schemas/ErrorResponse'

components:
  schemas:
    UploadRequest:
      type: object
      description: Body for requesting a file upload URL.
      required:
        - fileName
        - flowType
        - checksum
      properties:
        fileName:
          type: string
          description: The name of the file to be uploaded.
          example: "transactions_20260308.csv"
        flowType:
          type: string
          description: The business process category for which the file is being uploaded.
          example: "NAV"
        checksum:
          type: string
          description: The MD5 or SHA256 checksum of the file for integrity verification.
          example: "d41d8cd98f00b204e9800998ecf8427e"

    UploadResponse:
      type: object
      description: Response containing the fileId and the pre-signed URL for upload.
      properties:
        fileId:
          type: string
          format: uuid
          description: A unique identifier for this file upload process.
          example: "123e4567-e89b-12d3-a456-426614174000"
        uploadUrl:
          type: string
          format: uri
          description: The secure, time-limited URL to which the client should upload the file.
          example: "https://storage.googleapis.com/cams-quarantine-bucket/..."

    ErrorResponse:
      type: object
      properties:
```

```yaml
    timestamp:
      type: string
      format: date-time
    status:
      type: integer
    error:
      type: string
    message:
      type: string
    path:
      type: string
```

Specification [specs/001-secure-file-upload/spec.md]

Feature Specification: US-101: Secure File Upload Initiation

Feature Branch: 001-secure-file-upload **Created**: 2026-03-08 **Status**: Draft **Input**: User description: "US-101: Secure File Upload Initiation"

User Scenarios & Testing *(mandatory)*

User Story 1 - Securely Upload Large Financial Files (Priority: P1)

As a Portfolio Manager, I want to upload 1M+ record files via signed URLs so that files are securely received without size limits.

Why this priority: This is the foundational step for the entire file processing workflow. Without a secure and reliable upload mechanism, no other processing can occur. It directly impacts the core business function of ingesting financial data.

Independent Test: The upload process can be tested independently by verifying that a call to the API generates a valid pre-signed GCS URL. A file can then be uploaded to this URL using a standard HTTP client, and the system should correctly create an initial file record in the database with a status of AWAITING_UPLOAD, which can be queried via a status endpoint.

Acceptance Scenarios:

1. **Given** a Portfolio Manager is authenticated, **When** they make an API call to request an upload URL, providing the file name, file type (e.g., "NAV"), and a checksum (MD5/SHA256), **Then** the system returns a unique fileId and a valid, time-limited (24-hour expiry) pre-signed URL for uploading the file directly to a secure, isolated GCS bucket.

2. **Given** a valid pre-signed URL has been generated, **When** the user uploads a 4GB CSV file to that URL, **Then** the file is successfully stored in the designated GCS

"quarantine" bucket, and the API that generated the URL responds in under 200ms (p99).

3. **Given** a file has been successfully uploaded to the GCS bucket, **When** the system receives the notification of the new file, **Then** it transitions the corresponding file record in the database to UPLOADED.

Edge Cases

- **What happens when** an upload is attempted with an expired pre-signed URL? The system (GCS) should reject the upload with a 4xx error.
- **What happens when** the provided checksum during the request does not match the checksum of the uploaded file? The file should be moved to a "failed" bucket with a status of VALIDATION_FAILED and a reason of "Checksum mismatch".
- **How does the system handle** a request for an upload URL with a missing file name or flow type? The API should return a 400 Bad Request error with a descriptive message.
- **What happens if** the file upload fails midway? The incomplete file part should be automatically cleaned up by GCS bucket lifecycle policies. The system will not know about this file, as the completion trigger is never fired.

Requirements *(mandatory)*

Functional Requirements

- **FR-001**: The system MUST provide a secure API endpoint for clients to request a pre-signed URL for file uploads.
- **FR-002**: The pre-signed URL MUST be generated for a specific object name in a designated GCS "quarantine" bucket.
- **FR-003**: The pre-signed URL MUST have a configurable expiration time (defaulting to 24 hours).
- **FR-004**: The system MUST create a preliminary file record in a persistent database (Spanner) when the upload URL is requested, including metadata like file name and flow_type.
- **FR-005**: The system MUST support CSV and JSON file types up to 5GB.
- **FR-006**: The initial API response for the pre-signed URL request MUST be faster than 200ms (p99).
- **FR-007**: Raw uploaded files MUST be stored in an isolated GCS bucket with a Time-To-Live (TTL) policy for eventual cleanup.

Key Entities

- **FileRecord**: Represents a file being processed.
    - **Attributes**: fileId (unique identifier), originalFileName, flowType, checksum, status, createdAt, updatedAt.
- **UploadRequest**: Represents the client's request to upload a file.
    - **Attributes**: fileName, flowType, checksum.

Success Criteria *(mandatory)*

Measurable Outcomes

- **SC-001**: 100% of file upload requests result in the generation of a valid, secure, pre-signed URL.
- **SC-002**: The system successfully ingests files up to 5GB in size without data loss or corruption.
- **SC-003**: The API endpoint for generating signed URLs maintains a p99 latency of less than 200ms under a load of 1000 concurrent requests.
- **SC-004**: 99.99% of valid files uploaded to the pre-signed URL are successfully stored in the GCS quarantine bucket and have a corresponding record created in the database.

Research file [specs/001-secure-file-upload/research.md]

Phase 0: Research - US-101: Secure File Upload Initiation

Purpose: Resolve unknowns identified during planning before starting design. **Created**: 2026-03-08 **Plan**: plan.md

No "NEEDS CLARIFICATION" items were identified in the Technical Context, as the project constitution.md provided a clear technology stack.

This phase will focus on confirming best practices for the selected technologies.

Research Tasks

1. **GCP Pre-Signed URL Generation with Spring Boot**
    - **Task**: Investigate the best practices for generating GCS V4 pre-signed URLs using the spring-cloud-gcp-starter-storage library.
    - **Goal**: Confirm the necessary IAM permissions and service account configuration required for the microservice to

grant storage.objects.create permission to a client for a limited time. Document the core Java methods and classes involved.

2. **Spanner Integration with Spring Data**
    - **Task**: Research the optimal schema design for the FileRecord entity in Google Cloud Spanner, considering future query patterns (e.g., querying by status, flow type).
    - **Goal**: Define the DDL for the FileRecord table, including primary key selection (e.g., fileId as a UUID string) and any secondary indexes. Confirm the correct Spring Data Spanner annotations (@Table, @PrimaryKey, @Column).
3. **Local Development with Testcontainers**
    - **Task**: Find best practices for setting up a local development environment for Spanner and Pub/Sub using Testcontainers.
    - **Goal**: Provide a docker-compose.yml or Testcontainer configuration example for spinning up the Spanner emulator and Pub/Sub emulator. Document how to configure Spring Boot to connect to these emulators during integration tests.

DataModel [specs/001-secure-file-upload/data-model.md]

Phase 1: Data Model - US-101: Secure File Upload Initiation

Purpose: Define the data entities required for this feature. **Created**: 2026-03-08 **Plan**: plan.md

This document outlines the structure of the data entities based on the feature specification.

Entity: FileRecord

Represents the state and metadata of a single file throughout its processing lifecycle. This entity is persisted in Google Cloud Spanner.

Table Name: file_records

Attribute	Data Type (Spanner)	Description	Constraints / Rules
fileId	STRING(36)	**Primary Key**. A unique identifier (UUID) for the file record, generated by the service.	NOT NULL

Attribute	Data Type (Spanner)	Description	Constraints / Rules
originalFileName	STRING(1024)	The original name of the file provided by the user.	NOT NULL
flowType	STRING(255)	The business process category (e.g., "NAV", "Transactions"). Used to select validation templates.	NOT NULL
checksum	STRING(255)	The client-provided checksum (e.g., MD5 or SHA256) for integrity validation post-upload.	NOT NULL
status	STRING(50)	The current state of the file in the processing workflow. Initial status is AWAITING_UPLOAD.	NOT NULL
createdAt	TIMESTAMP	Timestamp when the record was created (when the upload URL was requested).	NOT NULL
updatedAt	TIMESTAMP	Timestamp of the last modification to the record.	NOT NULL
gcsBucket	STRING(1024)	The GCS bucket where the file is stored.	
gcsObjectPath	STRING(2048)	The full path to the file object within the GCS bucket.	

Spanner DDL

```
CREATE TABLE file_records (
  fileId STRING(36) NOT NULL,
  originalFileName STRING(1024) NOT NULL,
  flowType STRING(255) NOT NULL,
  checksum STRING(255) NOT NULL,
  status STRING(50) NOT NULL,
  createdAt TIMESTAMP NOT NULL OPTIONS (allow_commit_timestamp=true),
  updatedAt TIMESTAMP NOT NULL OPTIONS (allow_commit_timestamp=true),
```

```
  gcsBucket STRING(1024),
  gcsObjectPath STRING(2048),
) PRIMARY KEY (fileId);

CREATE INDEX file_records_by_status ON file_records(status);
```

Plan [specs/001-secure-file-upload/plan.md]

Implementation Plan: US-101: Secure File Upload Initiation

Branch: 001-secure-file-upload | **Date**: 2026-03-09 | **Spec**: spec.md **Input**: Feature specification from /specs/001-secure-file-upload/spec.md

Note: This template is filled in by the /speckit.plan command.
See .specify/templates/plan-template.md for the execution workflow.

Summary

This plan outlines the work to implement US-101, a secure file upload initiation mechanism. The core of this feature is a Spring WebFlux API endpoint that generates a time-limited, pre-signed Google Cloud Storage (GCS) URL. This allows clients to upload large files (up to 5GB) directly and securely to a designated "quarantine" bucket. Upon URL generation, a preliminary record is created in a Cloud Spanner database to track the file's lifecycle, starting with an AWAITING_UPLOAD status.

Technical Context

Language/Version: Java 21 **Primary Dependencies**: Spring Boot 3.x, Spring WebFlux, Spring Cloud GCP (Storage), OpenAPI 3.0, JUnit 5 **Storage**: Google Cloud Storage (GCS) for files, Cloud Spanner for metadata **Testing**: JUnit 5 **Target Platform**: Google Kubernetes Engine (GKE) **Project Type**: Web-service **Performance Goals**: < 200ms (p99) latency for the pre-signed URL generation API under 1000 concurrent requests. **Constraints**: Support for files up to 5GB, pre-signed URLs must expire (default 24 hours). **Scale/Scope**: Initial ingress point for all file processing, must handle large financial files (1M+ records).

Constitution Check

GATE: Must pass before Phase 0 research. Re-check after Phase 1 design.

- **[PASS] Event-Driven First**: The feature correctly implements the initial step of an event-driven workflow. It avoids handling file streams directly in the API and prepares for a subsequent event-based trigger upon file arrival in GCS.

- **[PASS] Asynchronous Processing is Mandatory**: The use of pre-signed URLs for direct-to-GCS uploads is a fully asynchronous pattern, aligning perfectly with the constitution. The API's responsibility ends after providing the URL.
- **[PASS] Horizontal Scalability by Design**: The API endpoint is stateless, making it suitable for horizontal scaling on GKE as mandated.
- **[PASS] Reliability Through Idempotency and State**: The plan includes creating a file record in Spanner at the start, which is the first step in tracking state through the finite state machine (FSM).
- **[PASS] Spec-Driven Development**: The feature will be driven by an OpenAPI specification, which will be created in Phase 1.

Project Structure

Documentation (this feature)

```
specs/001-secure-file-upload/
 ├── plan.md            # This file (/speckit.plan command output)
 ├── research.md        # Phase 0 output (/speckit.plan command)
 ├── data-model.md      # Phase 1 output (/speckit.plan command)
 ├── quickstart.md      # Phase 1 output (/speckit.plan command)
 ├── contracts/         # Phase 1 output (/speckit.plan command)
 └── tasks.md           # Phase 2 output (/speckit.tasks command - NOT created by
/speckit.plan)
```

Source Code (Repository Root)

```
# Option 1: Single project (DEFAULT)
src/
 ├── main/
 |    ├── java/
 |    |    └── com/
 |    |        └── cams/
 |    |            └── fileprocessing/
 |    |                ├── CamsFileProcessingServiceApplication.java
 |    |                ├── common/
 |    |                ├── config/
 |    |                ├── features/
 |    |                |    └── upload/
 |    |                |        ├── UploadController.java
 |    |                |        ├── UploadService.java
 |    |                |        ├── FileRecordRepository.java
```

```
│   │           │       └── models/
│   │           │           └── FileRecord.java
│   │           └── gcp/
│   │               └── GcsService.java
│   └── resources/
│       ├── application.yml
│       └── logback-spring.xml
└── test/
    └── java/
        └── com/
            └── cams/
                └── fileprocessing/
                    └── features/
                        └── upload/
                            ├── UploadControllerTest.java
                            └── UploadServiceTest.java
```

Structure Decision: A standard single-project Spring Boot structure is chosen. The core logic for the secure upload feature will be encapsulated within src/main/java/com/cams/fileprocessing/features/upload. This provides clear separation of concerns while remaining simple for the initial service.

Tasks [specify/001-secure-file-upload/tasks.md]

Tasks: US-101: Secure File Upload Initiation

Input: Design documents from /specs/001-secure-file-upload/ **Prerequisites**: plan.md (required), spec.md (required for user stories), research.md, data-model.md, contracts/

Tests: The examples below include test tasks. Tests are OPTIONAL - only include them if explicitly requested in the feature specification.

Organization: Tasks are grouped by user story to enable independent implementation and testing of each story.

Format: [ID] [P?] [Story] Description

- **[P]**: Can run in parallel (different files, no dependencies)
- **[Story]**: Which user story this task belongs to (e.g., US1, US2, US3)
- Include exact file paths in descriptions

Path Conventions

- **Single project**: src/, tests/ at repository root
- **Web app**: backend/src/, frontend/src/
- **Mobile**: api/src/, ios/src/ or android/src/
- Paths shown below assume single project - adjust based on plan.md structure

Phase 1: Setup (Shared Infrastructure)

Purpose: Project initialization and basic structure

- [X] T001 Create the project structure in src/main and src/test as defined in plan.md
- [X] T002 Initialize a Spring Boot project with Java 21, Maven, and add primary dependencies: Spring WebFlux, Spring Cloud GCP (Storage), and Spring Data for Spanner in pom.xml
- [X] T003 [P] Configure basic logging in src/main/resources/logback-spring.xml
- [X] T004 [P] Configure base application properties for a local profile in src/main/resources/application.yml

---

Phase 2: Foundational (Blocking Prerequisites)

Purpose: Core infrastructure that MUST be complete before ANY user story can be implemented

⚠️ **CRITICAL**: No user story work can begin until this phase is complete

- [X] T005 Setup Spanner emulator and Pub/Sub emulator configuration for local testing, potentially using Testcontainers.
- [X] T006 [P] Implement a generic GCP configuration class in src/main/java/com/cams/fileprocessing/config/GcpConfig.java to manage GCP credentials and project ID.
- [X] T007 [P] Implement a base exception handler for REST APIs in src/main/java/com/cams/fileprocessing/common/GlobalExceptionHandler.java to standardize error responses.
- [X] T008 Create the main application class src/main/java/com/cams/fileprocessing/CamsFileProcessingServiceApplication.java.

Checkpoint: Foundation ready - user story implementation can now begin in parallel

---

Phase 3: User Story 1 - Securely Upload Large Financial Files (Priority: P1)🎯 MVP

Goal: As a Portfolio Manager, I want to upload 1M+ record files via signed URLs so that files are securely received without size limits.

Independent Test: An API call generates a valid GCS pre-signed URL. A file can be uploaded to this URL, and a FileRecord is created in the database with status AWAITING_UPLOAD.

Tests for User Story 1⚠️

NOTE: Write these tests FIRST, ensure they FAIL before implementation

- [ ] T009 [P] [US1] Write a contract test for the requestUploadUrl endpoint in src/test/java/com/cams/fileprocessing/features/upload/UploadControllerTest.java to verify the API against contracts/openapi.yaml.
- [ ] T010 [P] [US1] Write an integration test in src/test/java/com/cams/fileprocessing/features/upload/UploadServiceTest.java to ensure the service correctly interacts with the GCS and Spanner emulators.

Implementation for User Story 1

- [ ] T011 [P] [US1] Create the FileRecord entity based on data-model.md in src/main/java/com/cams/fileprocessing/features/upload/models/FileRecord.java.
- [ ] T012 [P] [US1] Create the Spring Data repository interface for the FileRecord entity in src/main/java/com/cams/fileprocessing/features/upload/FileRecordRepository.java.
- [ ] T013 [US1] Implement the GcsService to generate pre-signed V4 URLs in src/main/java/com/cams/fileprocessing/gcp/GcsService.java.
- [ ] T014 [US1] Implement the UploadService to orchestrate the creation of the FileRecord and generation of the pre-signed URL in src/main/java/com/cams/fileprocessing/features/upload/UploadService.java.
- [ ] T015 [US1] Implement the UploadController with the requestUploadUrl endpoint defined in contracts/openapi.yaml in src/main/java/com/cams/fileprocessing/features/upload/UploadController.java.
- [ ] T016 [US1] Add input validation for the UploadRequest object in the UploadController or UploadService.

- [ ] T017 [US1] Add structured logging for all operations within the upload feature.

Checkpoint: At this point, User Story 1 should be fully functional and testable independently

---

Phase 4: Polish & Cross-Cutting Concerns

Purpose: Improvements that affect multiple user stories

- [ ] T018 [P] Add comprehensive Javadoc to all new public classes and methods.
- [ ] T019 [P] Enhance unit test coverage for services and utilities.
- [ ] T020 Review and refine security rules for GCS buckets and IAM permissions for the service account.
- [ ] T021 [P] Configure a lifecycle policy on the GCS "quarantine" bucket to automatically delete incomplete or old uploads (FR-007).
- [ ] T022 Validate the entire workflow described in quickstart.md to ensure it works end-to-end.

Dependencies & Execution Order

Phase Dependencies

- **Setup (Phase 1)**: No dependencies - can start immediately.
- **Foundational (Phase 2)**: Depends on Setup completion - BLOCKS all user stories.
- **User Story 1 (Phase 3)**: Depends on Foundational phase completion.
- **Polish (Phase 4)**: Depends on User Story 1 completion.

User Story Dependencies

- **User Story 1 (P1)**: Can start after Foundational (Phase 2) is complete.

Within Each User Story

- Tests (T009, T010) MUST be written and FAIL before implementation.
- Models (T011) and Repository (T012) before Services (T013, T014).
- Services (T013, T014) before Controller (T015).
- Core implementation before validation and logging (T016, T017).

Parallel Opportunities

- **Setup**: T003 and T004 can run in parallel.
- **Foundational**: T006 and T007 can run in parallel.
- **User Story 1**:
    - T009 and T010 (Tests) can be developed in parallel.
    - T011 and T012 (Model/Repo) can be developed in parallel with T013 (GcsService).

Parallel Example: User Story 1

 # Launch all tests for User Story 1 together:

Task: "T009 [P] [US1] Write a contract test for the requestUploadUrl endpoint..."

Task: "T010 [P] [US1] Write an integration test for the UploadService..."

 # Launch parallel implementation tasks after tests are failing:

Task: "T011 [P] [US1] Create the FileRecord entity..."

Task: "T012 [P] [US1] Create the Spring Data repository interface..."

Task: "T013 [US1] Implement the GcsService..."

Implementation Strategy

MVP First (User Story 1 Only)

1. Complete Phase 1: Setup
2. Complete Phase 2: Foundational (CRITICAL - blocks all stories)
3. Complete Phase 3: User Story 1
4. **STOP and VALIDATE**: Test User Story 1 independently using the steps in quickstart.md.
5. Deploy/demo if ready.

Incremental Delivery

1. Complete Setup + Foundational → Foundation ready.
2. Add User Story 1 → Test independently → Deploy/Demo (MVP!).

3. Each subsequent story adds value without breaking previous stories.

Questions to Siva

- Assumption: The sources from which we will get the files are
  - SFTP
  - UI->API
  - Storage Transfer Service [S3->GCB, URL->GCB, Azure Blob->GCB]

  Is the assumption correct?

- Based on the problem statement it could be a very Simple/Complex/Somewhere in between use that decides how complex the system has to be build for the current and the future use case and not over-engineer the solution. It will be good to have the discussion and understand more before we start implementing this solution using Github Speckit.
- Should we also include all the application, infrastructure and cyber security scaffolding [Authentication, Authorization, Logging, Exception Handling, trace and span id implementation [service communication using Istio] also in the case study.